

Online Supplementary Materials: A Communication-Efficient Decentralization Scheme: Insights from Recommender Systems

1. H&M Personalized Fashion Recommendations Dataset

The H&M dataset is provided by the H&M Group.¹ The transaction data spans from September 20th, 2018 to September 22nd, 2020, and contains 31,788,324 rows, each representing a customer’s purchase of a specific article. We process this data in three stages: domain filtering, temporal train/test splitting, and participant sampling.

The H&M Personalized Fashion Recommendations dataset provides customer transaction logs alongside customer and article metadata, where each transaction records a customer–article purchase event with a date.

Domain filtering. First, articles are restricted to the six most popular product types by unique product-code count. Second, only customers who have purchased at least distinct products across the full dataset are retained. Third, customers are restricted to those located in the 5,000 most common postal codes (excluding the single most common, which is a catch-all code). After filtering, each transaction is reduced to a (`customer_id`, `product_code`, `bought`) record where `bought` records the purchase-frequency count for each customer–article pair. The merged dataset contains approximately 116,000 interactions across 1,760 users and 8,618 items.

Aggregation. Transactions are aggregated to unique (customer, product) pairs with purchase count as the target signal. This is the natural unit for matrix factorization: the model learns one preference score per observed pair rather than one per purchase event.

Item filtering. Articles appearing in fewer than 10 interactions across the full merged dataset are removed. This retains 2,881 items and eliminates long-tail articles that carry insufficient signal for collaborative filtering.

User selection. All 1,760 users present in the source data are retained; no minimum-interaction threshold or top- K selection is applied at this stage.

Train / test split. The filtered interactions are split into training (80%) and test (20%) sets using a stratified random split with `random_state=1`, where stratification is by `user_id`. This ensures every user contributes rows to both partitions. No separate validation split is produced by this notebook; downstream notebooks derive a validation set from the training partition via a further 80/20 random split.

Cold-start. User cold-start is prevented by construction via the stratified split. Item cold-start is not explicitly prevented at this stage: because the split is random rather than per-user vocabulary-aware, test interactions may reference items absent from the training set.

2. Decentralization for a Deep-Learning-Based Recommender System

We introduce a Generalized Singular Value Decomposition (SVD) architecture that allows user- and item-specific latent factors to be trained by two separate deep neural networks (i.e., a two-tower

¹ <https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations>

Table 1 H&M dataset statistics after sampling and splitting.

Property	Value
<i>Item filter</i>	
Min. interaction count per item	10
Items retained	2,881
<i>Users</i>	
Users retained	1,760 (all)
Selection rule	All users from source data
<i>Split parameters</i>	
Test fraction	20% (random, stratified by user)
Validation fraction	20% (train/test only)
<i>Dataset scale (after splitting)</i>	
Users	1,760
Items (vocabulary)	2,881
Total interactions	~116,000
Training interactions	~93,000 (80%)
Test interactions	~23,000 (20%)
Rating scale	1–22 (purchase frequency count)
Matrix density	~2.3%
<i>Cold-start</i>	
User cold-start	None (stratified split)
Item cold-start	Possible (random split)

model). See Figure A1 for an illustration. We further demonstrate that the proposed decentralized recommender system scheme can be applied.

Our proposed Generalized SVD method is an extended architecture inspired by Matrix Factorization, where user- and item-specific latent factors are trained via neural networks than direct projection from the data. Let $\mathbf{p}_i^0$ and $\mathbf{q}_j^0$ be the first embedding layer which is converted directly from user- and item-level one-hot encoding. We can then employ two distinct layers (e.g., two rectified linear units) to transform $\mathbf{p}_i^0$ and $\mathbf{q}_j^0$ into $\mathbf{p}_i^1$ and $\mathbf{q}_j^1$, respectively. This process is repeated for n layers until we obtain the final user representation $\mathbf{p}_i^n$, and item representation, $\mathbf{q}_j^n$. Finally, we apply the dot product to these two final representations to make an estimated prediction, that is,

$$y_{ij} = \mathbf{p}_i^n \cdot \mathbf{q}_j^n.$$

The decentralized Generalized SVD architecture adheres to the same principles as the Matrix Factorization, which itself is based on the SVD method. The key distinction is that the gradients are propagated not only from the embedding layer but also through subsequent representational layers. One important note is that the generalized SVD architecture still maintain separate user and item latent factor learning. In other words, the proposed decentralization scheme, once applied, would only share item-specific gradients and would not reveal any user-specific information. This guarantees the same level of data privacy protection as we have for the matrix factorization method in the main text.

When new observations are generated, the model architecture obtains the final user representation, $\mathbf{p}_i^n$, and the specific item representation for this user, $\mathbf{q}_{ij}^n$. Here $\mathbf{q}_{ij}^n$ can be different across different users due to users may receive different gradients across the network. Backpropagation is

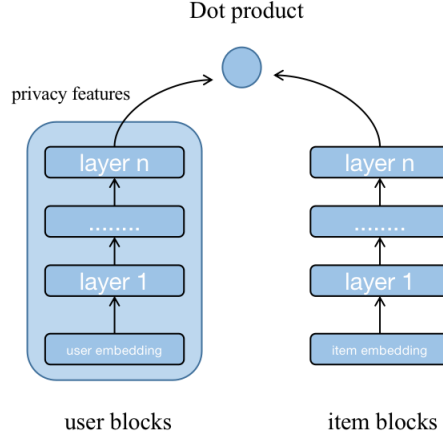


Figure A1 Generalized SVD Architecture

then performed after generating the final prediction. Mathematically, the gradients are calculated as follows

$$\begin{aligned}\nabla \mathbf{p}_i^n &= -\mathbf{q}_{ij}^n (Y_{ij} - \mathbf{p}_i^n \mathbf{q}_{ij}^n) + \lambda \mathbf{p}_i^n, \\ \nabla \mathbf{q}_{ij}^n &= -\mathbf{p}_i^n (Y_{ij} - \mathbf{p}_i^n \mathbf{q}_{ij}^n) + \lambda \mathbf{q}_{ij}^n.\end{aligned}$$

Following backpropagation, gradients for each representation layer are obtained, including $\nabla \mathbf{p}_i^0, \nabla \mathbf{p}_i^1, \dots, \nabla \mathbf{p}_i^n$ and $\nabla \mathbf{q}_{ij}^0, \nabla \mathbf{q}_{ij}^1, \dots, \nabla \mathbf{q}_{ij}^n$. Subsequently, a gradient descent method is applied to update the local model parameters. To ensure user privacy, only the gradients $\nabla \mathbf{q}_{ij}^0, \nabla \mathbf{q}_{ij}^1, \dots, \nabla \mathbf{q}_{ij}^n$ are shared with neighboring users, while $\nabla \mathbf{p}_i^0, \nabla \mathbf{p}_i^1, \dots, \nabla \mathbf{p}_i^n$ are kept confidential for a specific user. Ultimately, neighbors receive the gradients from one another to update their local model parameters layer by layer, that is,

$$\begin{aligned}\mathbf{q}_{ij}^n &= \mathbf{q}_{ij}^n - \eta \cdot \nabla \mathbf{q}_{ij}^n, \\ \mathbf{q}_{ij}^{n-1} &= \mathbf{q}_{ij}^{n-1} - \eta \cdot \nabla \mathbf{q}_{ij}^{n-1}, \\ &\dots \\ \mathbf{q}_{ij}^0 &= \mathbf{q}_{ij}^0 - \eta \cdot \nabla \mathbf{q}_{ij}^0.\end{aligned}$$

In our numerical experiment, we utilize two representation layers which results in a much larger model than that of simple Matrix Factorization (MF). Due to its size, it does take considerably longer than MF for decentralized learning, and is therefore more difficult to perform hyperparameter tuning. We extracted 100 users from the MovieLens 100K dataset and subsequently applied our proposed generalized SVD architecture. The experimental results demonstrate that the scheme adheres to consistent rules and surpasses the traditional SVD method by 55.4% in terms of performance.

3. More on Order Selection in A Communication Network

3.1. Order selection

In the rest of this section, we discuss how to select the order K . The goal is to balance the tradeoff between a large communication cost and unsatisfactory learning results. To understand this tradeoff, recall that K determines how far a participant's gradients can reach within a given network. Selecting a small K can expedite information propagation across the network, thereby reducing communication cost. However, a small K may lead to non-convergence and decreased

accuracy. On the other hand, choosing a large K can demand high computational resources and data transmission bandwidth, but may lead to better convergence results. In a connected graph, for example, when all nodes are reachable from any other nodes, the proposed method with a sufficiently large K (e.g., K being the diameter, the longest distance between any pair of nodes) is expected to have the same performance as if all data were merged at a center (i.e., central learning). This is because all participants can receive all gradients in every round.

To determine the optimal order K we introduce an order selection method for efficient gradient information transfer. Unlike traditional cross-validation techniques, our proposed order selection method offers an efficient approach to circumvent intensive computation. First, we note that as the order K increases, the number of connected users also increases. However, the increase rate may vary, depending on the underlying network. Furthermore, there is a limit on how many connected users are possible. In the undirected case, the limit is equal to the number of edges in a fully connected graph, but in the directed case, the limit depends on the graph structure and may stop increasing well before the fully connected value is reached. We notice that the derivative (i.e., forward difference) indicates the rate of change with respect to the number of connected users. Therefore, we select K as the point where the second derivative of the number of edges reaches its minimum value. In other words, this includes a large number of participants without drastically increasing the order, which provides the best cost effectiveness in terms of model performance and communication overhead.

3.2. A Toy Example on Order Selection

The number of sharing edges with respect to a specific order is defined as the total number of connections through which the participants transfer the model information in a given communication network. For example, with a communication network denoted by its adjacency matrix A , if we take the first-order sharing, participant i 's gradients will be sent to its neighbors $\mathcal{N}_i$, and the number of sharing edge for participant i is $|\mathcal{N}_i| = \sum_{j=1}^M A_{ij}$, where $|\cdot|$ denotes the cardinality norm of a set (i.e., total number of elements). Thus, the total number of sharing edges is $E_A(1) = \sum_{i=1}^M |\mathcal{N}_i| = \sum_{i=1}^M \sum_{j=1}^M A_{ij}$, which is just the total number of edges in network A . Let's now consider the second-order sharing, by which participant i can transfer its model information to its neighbors and its neighbors of neighbors, that is, $i \rightarrow j \rightarrow j'$, if $A_{ij} = A_{jj'} = 1$. Hence, the number of sharing edges for participant i is now $E_A(2) = \sum_{j=1}^M A_{ij} + \sum_{j=1}^M \sum_{j' \neq i}^M A_{ij} A_{jj'}$, and the total number of sharing edges is $\sum_{i=1}^M \{ \sum_{j=1}^M A_{ij} + \sum_{j=1}^M \sum_{j' \neq i}^M A_{ij} A_{jj'} \}$. We can calculate the case for any k th-order sharing analogously. In particular, if taking the maximum order and the network A is a connected graph (i.e., there is always a path connecting two arbitrary nodes), then the total number of sharing edges will be just $E_A(max) = M(M-1)$, the same as a fully connected graph.

Next, we provide a toy example, using a 10-node network to further illustrate the statistics discussed above. The first-order derivative $DE_A(k) = E_A(k) - E_A(k-1)$ denotes the increased number of connections when the sharing order increases from $k-1$ to k , while the second-order derivative $D^2E_A(k) = DE_A(k) - DE_A(k-1)$ evaluates the speed of this edge-increasing trend. Those statistics are demonstrated in Figures A3 and A2 and Table 2 below. Intuitively, it is desirable for each increase in communication order to yield a substantial number of new connections beyond those in the original network. This property is particularly beneficial in large-scale systems with many participants and sparse communication topologies, where expanding connectivity can significantly enhance information diffusion and potentially model convergence. We may stop increasing the order once the marginal increase in the number of sharing edges becomes negligible. Note that this order selection procedure only applies to the communication-constraint scenario, that is, the between-participant connections are limited by a constrained network structure, which is irrelevant to the algorithm. If the network is artificially designed and embedded in the underlying algorithm, there is generally no need for order selection. (Please refer to Section 4 of the paper for the definitions of constrained and designed networks in communication topology.)

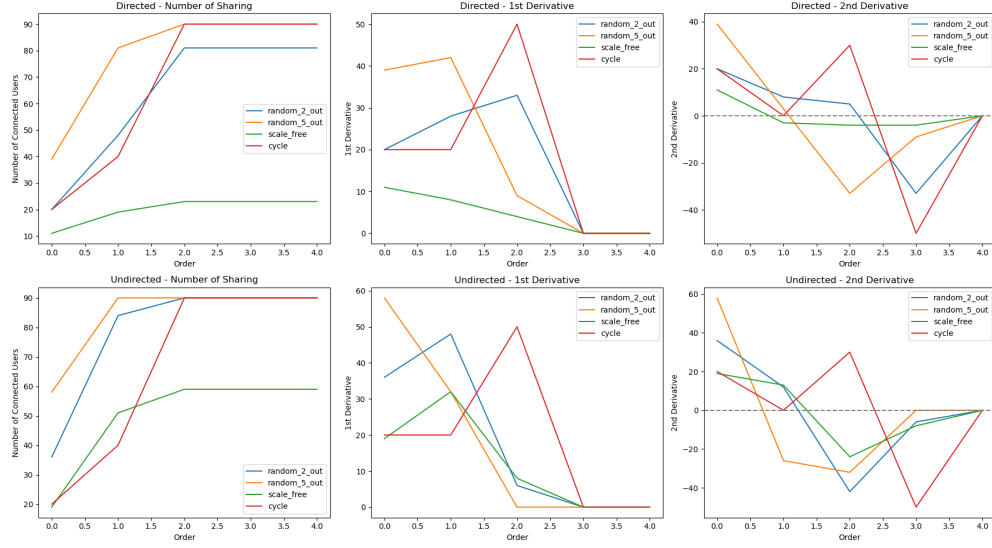


Figure A2 Statistics of Sharing Edges in the Toy Example with 10 Nodes.

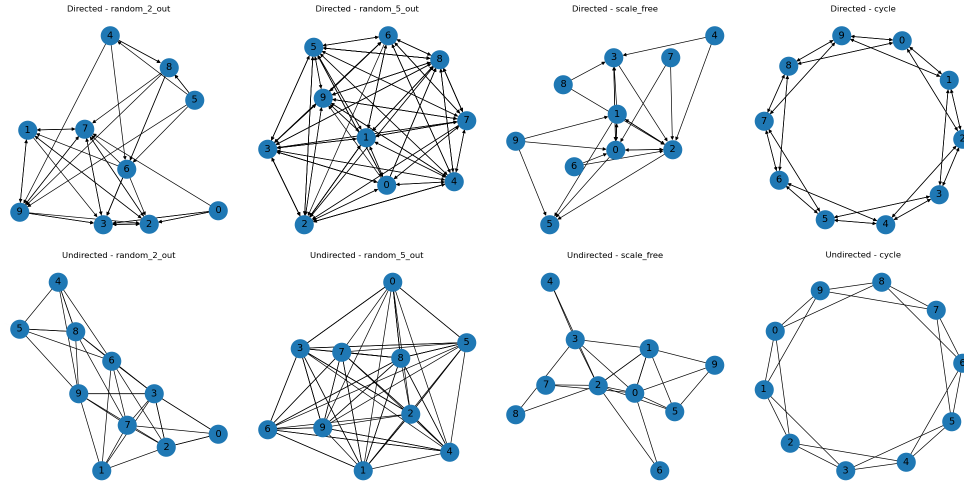


Figure A3 Different Network Structures in the Toy Example with 10 Nodes.

Table 2 A summary of the total number of sharing edges over sharing orders for the toy example.

Order	Total Sharing Edges					1st Derivative					2nd Derivative				
	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5
Directed															
Random-2-out	20	48	81	81	81	0	28	33	0	0	0	28	5	-33	0
Random-5-out	39	81	90	90	90	0	42	9	0	0	0	42	-33	-9	0
Scale-free	11	19	23	23	23	0	8	4	0	0	0	8	-4	-4	0
Cycle	20	40	90	90	90	0	20	50	0	0	0	20	30	-50	0
Undirected															
Random-2-out	36	84	90	90	90	0	48	6	0	0	0	48	-42	-6	0
Random-5-out	58	90	90	90	90	0	32	0	0	0	0	32	-32	0	0
Scale-free	19	51	59	59	59	0	32	8	0	0	0	32	-24	-8	0
Cycle	20	40	90	90	90	0	20	50	0	0	0	20	30	-50	0

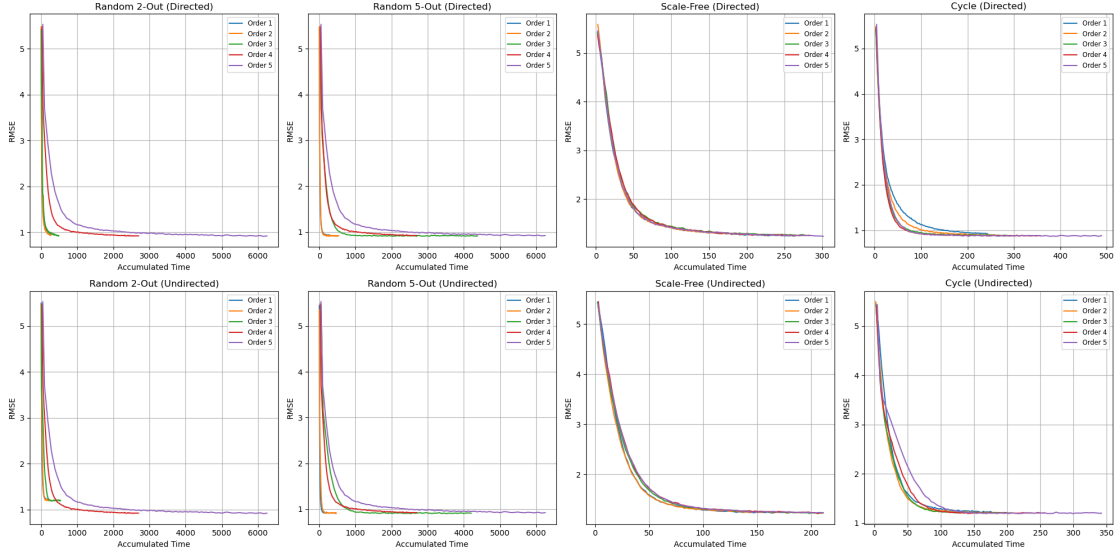


Figure A4 Comparing performance of various orders of the proposed FOAF method over four different networks with directed (top) and undirected settings (bottom).

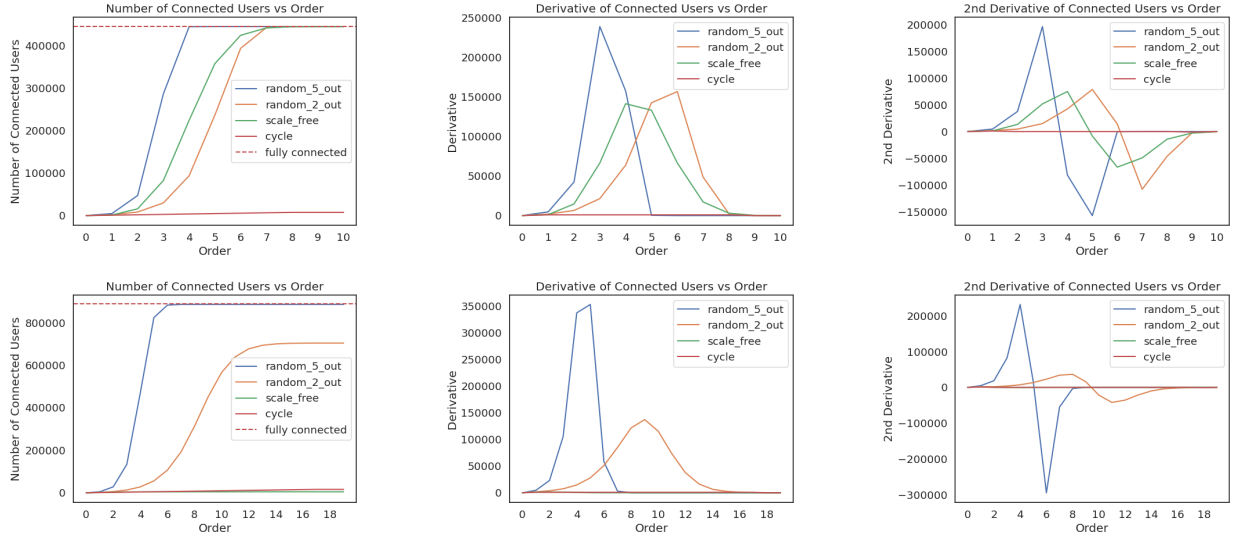


Figure A5 Order selection for undirected graph types (top) and directed graph types (bottom), where the order with the smallest second derivative value is selected.

3.3. Sensitivity Analysis with respect to the Order

In the undirected networks, the proposed method with the optimal order usually achieves accuracy better than the 1st-order counterpart, but worse than the case when the max-order is utilized. In the directed networks, this pattern becomes less clear. Figure A4 compares the validation loss curves of different orders of our method. We see that as the order increases, the performance tends to improve as well. However, this often comes at the cost of training time. One interesting thing to note is that the loss curve particularly for the optimal order decreases inconsistently. One possible reason is that the optimal order is often greater than 3, which leads to a higher computational cost than those with order 1 or 2. This can be seen from Figure A5, where, following Section 3.1 of this Online Supplementary Materials, the order with the smallest second derivative value is selected.

4. List of Competing Methods

In our experiments in the main text, we compare the proposed method with 12 state-of-the-art centralized and decentralized federated learning algorithms for recommender systems reported in the existing literature, encompassing both centralized and decentralized paradigms. The competing methods can roughly be classified into four categories:

1. **Local** – Methods that do not allow communications or data aggregation. Each participant updates their own model.

2. **Data-Centralized Learning (CL)** – Methods that require all data to be aggregated on a central server prior to training.

3. **Central-Server-Based Federated Learning (CFL)** – Methods designed for the classical federated learning setting, where data remain local but a central server coordinates model updates.

4. **Fully Decentralized Federated Learning (DFL)** – Methods designed for decentralized federated learning, where no central server is involved and coordination occurs directly among participants.

The full list of baseline methods is provided below, where the abbreviations in the parentheses indicate the category each method belongs to: data centralized learning (CL), central-server-based federated learning (CFL), or decentralized federated learning (DFL):

1. **GL** (DFL): Gossip Learning [4], a decentralized learning method where models are updated locally and exchanged peer-to-peer, including a local model aggregation based on a pre-specified mixing matrix.

2. **DMF** (DFL): Decentralized Matrix Factorization [2], a collaborative filtering approach where users or nodes update and share low-rank matrix factors locally through peer-to-peer communication, without relying on a central server.

3. **FCF** (CFL): Federated Collaborative Filtering [1], the first MF-based federated recommendation method.

4. **FedNCF** (CFL): Federated Neural Collaborative Filtering [9, (NCF)], each participant trains local NCF models using its private data and then submits updates related to global parameters to a central server.

5. **FedGNN** (CFL): Federated Graph Neural Networks [10], a federated recommendation method that leverages graph neural networks (GNNs) to model user-item interactions while preserving user privacy across distributed participants.

6. **RFR** (CFL): Regularized Federated Recommendation [6], a federated recommendation method that reformulates matrix factorization as a convex optimization problem, ensuring convergence while balancing global model and local personalization.

7. **RFR** (CFL): A communication-efficient extension of RFR [6] that accelerates convergence in federated recommendation by selectively sampling participant updates, reducing communication rounds while preserving accuracy and robustness.

8. **FedAvg** (CFL): Federated Averaging [7], a canonical federated learning scheme, where the central server updates the global model parameters, and shares the updated parameters back to participants. A simple average across randomly chosen local models is used for combining the participant models into the global model.

9. **CMF** (CL): Central Matrix Factorization [3], a standard MF model based on all the data collected at a central server.

10. **CPMF** (CL): Probabilistic Matrix Factorization [8], a probabilistic MF model using all data collected by a central server.

11. **CSVD++** (CL): A popular variant of SVD [5] considers user and item biases to enhance recommendation performance.

12. **LL** (Local): The communication between nodes is not allowed. Each participant's model is trained on their local data only.

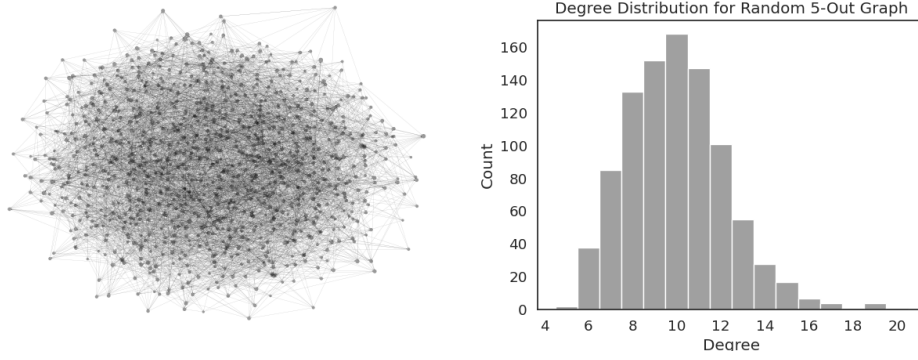


Figure A6 Random 5-Out Graph and Degree Distribution

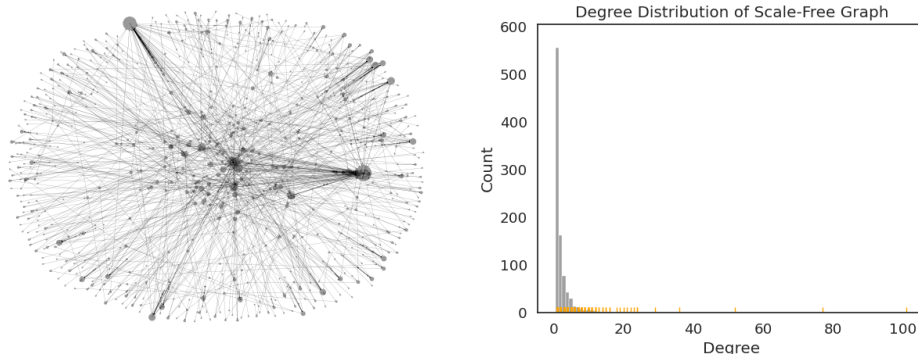


Figure A7 Scale-Free Graph and Degree Distribution

5. More Details on Communication Network Structure

5.1. Random k -Out Graph

The random k -out graph is a type of directed graph where each node has k out-degree and an unconstrained in-degree. Given a source node i , the simplest implementation has the target nodes $i_1, \dots, i_k$ selected uniformly at random. We utilize a weighted target node selection that includes a slight preferential attachment. This means that nodes with more in-degrees will have a higher probability of becoming a target node. In our experiments we use both a random 5-out graph and a random 2-out graph. Figure A6 provides an illustration of a random 5-out graph and its degree distribution in histogram.

5.2. Scale-Free Graph

Scale-free graphs exhibit a characteristic where a small subset of nodes has significantly higher connectivity compared to the remaining nodes, which exhibit lower connectivity. In this network, there are three controlled parameters. The first parameter, α , represents the probability of adding a new node that is connected to an existing node chosen randomly based on the in-degree distribution. The second parameter, β , controls the probability of adding an edge between two existing nodes. The last parameter, γ , determines the probability of adding a new node connected to an existing node chosen randomly based on the out-degree distribution. We use values of $\alpha = 0.5$, $\beta = 0.25$, and $\gamma = 0.25$. Similar to the random k -out graph, the undirected version of the scale-free graph is created by converting all the directed edges to be undirected. Figure A7 provides an illustration of a scale-free graph, as well as its degree distribution in histogram, where the gray bars and yellow bars denote the number of in-degrees and out-degrees, respectively.

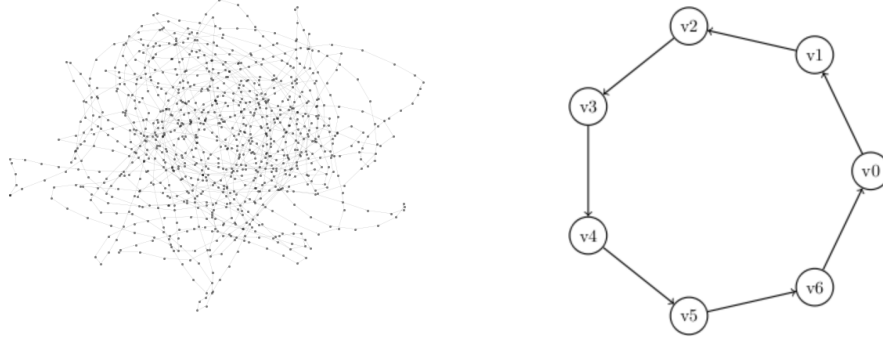


Figure A8 Cycle Graph and Simple Example

5.3. Cycle graph

The cycle graph depicts a configuration of cyclically connected participants. The left panel of Figure A8 shows the cycle graph used in our experiments, and the right panel shows a simple cycle graph for reference. We do not use the optimal order selection for the cycle graph as its specialized structure forces the number of increased node connections to grow linearly as the order increases. An undirected cycle graph can be created similar to the counterpart of random k -out and scale-free graphs, and has connections to both neighbors.

6. Additional Theoretical Results

6.1. Alternative Assumption

Assumption 4 (Alternative Assumption on Communication Network) Let $A_{M \times M} = (a_{ij})_{1 \leq i, j \leq M}$ be the adjacency matrix of a random communication network $\mathcal{G} = \{V, E\}$, through which each participant shares the gradient with its neighborhood in the proposed Algorithm 1. Let $A_{M \times M}^* = (a_{ij}^*)_{1 \leq i, j \leq M}$ be a square matrix, where $P(a_{ij} = 1) = a_{ij}^* \in [0, 1]$, and $a_{ij}^* = \omega_i$ as stated in Algorithm 1. Assume that A^* is symmetric and semi-positive definite, and denote its smallest non-zero eigenvalue as $\rho_{min} = \lambda_{min}^+(A^*) > 0$.

6.2. Assumption 5 and Proof of Theorem 2

Assumption 5 (Mini-batch gradient estimator for URS) At iteration t , let $\hat{\nabla}F(\tilde{\theta}^t, \xi^t)$ denote the mini-batch gradient estimator under the URS scheme. Conditional on $\mathcal{F}_t = \sigma(\tilde{\theta}^0, \xi^0, \dots, \xi^{t-1})$, assume

$$\mathbb{E}[\hat{\nabla}F(\tilde{\theta}^t, \xi^t) | \mathcal{F}_t] = \nabla F(\tilde{\theta}^t),$$

and

$$\mathbb{E}\left[\left\|\hat{\nabla}F(\tilde{\theta}^t, \xi^t) - \nabla F(\tilde{\theta}^t)\right\|^2 \middle| \mathcal{F}_t\right] \leq \nu(N_0),$$

where $\nu(N_0)$ is non-increasing in the mini-batch size N_0 and satisfies $\nu(N_0) = 0$ when the mini-batch equals the full local dataset.

A simple finite-sample form of $\nu(N_0)$ is:

$$\nu(N_0) = \sigma^2 \frac{1 - \frac{N_0}{n}}{N_0}, \quad 1 \leq N_0 \leq n,$$

where each client has the same local sample size n , sampling is without replacement, and σ^2 bounds the within-client gradient variance.

Theorem 2 (Convergence under URS mini-batch stochastic updates) *Suppose Assumptions 1–4 and Assumption 5 hold. Consider the stochastic update*

$$\tilde{\boldsymbol{\theta}}^{t+1} = \tilde{\boldsymbol{\theta}}^t - \eta_t \tilde{A}^* \hat{\nabla} F(\tilde{\boldsymbol{\theta}}^t, \xi^t), \quad \tilde{A}^* := A^* \otimes I_p.$$

Let $\|\mathbf{v}\|_{\tilde{A}^*}^2 = \mathbf{v}^\top \tilde{A}^* \mathbf{v}$, where $\tilde{A}^{*\dagger}$ is the Moore–Penrose inverse of $\tilde{A}^*$. Then the following one-step recursion holds:

$$\mathbb{E} \left[\|\tilde{\boldsymbol{\theta}}^{t+1} - \tilde{\boldsymbol{\theta}}^*\|_{\tilde{A}^*}^2 \mid \mathcal{F}_t \right] \leq \left(1 - 2\eta_t \mu \underline{\lambda}_A + \eta_t^2 L^2 \bar{\lambda}_A^2 \right) \|\tilde{\boldsymbol{\theta}}^t - \tilde{\boldsymbol{\theta}}^*\|_{\tilde{A}^*}^2 + \eta_t^2 \bar{\lambda}_A \nu(N_0).$$

Consequently, if $\eta_t \equiv \eta$ and

$$0 < \eta < \frac{2\mu \underline{\lambda}_A}{L^2 \bar{\lambda}_A^2},$$

then

$$\mathbb{E} \|\tilde{\boldsymbol{\theta}}^t - \tilde{\boldsymbol{\theta}}^*\|^2 \leq \frac{\bar{\lambda}_A}{\underline{\lambda}_A} \rho^t \|\tilde{\boldsymbol{\theta}}^0 - \tilde{\boldsymbol{\theta}}^*\|^2 + \frac{\eta \bar{\lambda}_A^2 \nu(N_0)}{2\mu \underline{\lambda}_A - \eta L^2 \bar{\lambda}_A^2},$$

where

$$\rho = 1 - 2\eta \mu \underline{\lambda}_A + \eta^2 L^2 \bar{\lambda}_A^2 \in (0, 1).$$

Thus, the iterates converge linearly to a neighborhood of $\tilde{\boldsymbol{\theta}}^*$ in mean-squared error, and the size of this mean-squared neighborhood is of order $O(\eta \nu(N_0))$.

If $\eta_t = \gamma/t$ with $2\gamma \mu \underline{\lambda}_A > 1$, then

$$\mathbb{E} \|\tilde{\boldsymbol{\theta}}^t - \tilde{\boldsymbol{\theta}}^*\|^2 = O\left(\frac{1 + \nu(N_0)}{t}\right).$$

When the mini-batch equals the full local dataset at each participant, $\nu(N_0) = 0$, and the stochastic update reduces to the deterministic update in Theorem 1.

Proof:

L et

$$\Delta^t = \hat{\nabla} F(\tilde{\boldsymbol{\theta}}^t, \xi^t) - \nabla F(\tilde{\boldsymbol{\theta}}^t).$$

By Assumption 5,

$$\mathbb{E}[\Delta^t \mid \mathcal{F}_t] = 0, \quad \mathbb{E}[\|\Delta^t\|^2 \mid \mathcal{F}_t] \leq \nu(N_0).$$

Define $e^t = \tilde{\boldsymbol{\theta}}^t - \tilde{\boldsymbol{\theta}}^*$. Since all iterates remain in the affine space Θ_{A^*} , we have $e^t \in \text{range}(\tilde{A}^*)$. The update gives

$$e^{t+1} = e^t - \eta_t \tilde{A}^* \nabla F(\tilde{\boldsymbol{\theta}}^t) - \eta_t \tilde{A}^* \Delta^t.$$

Taking the squared $\tilde{A}^{*\dagger}$ -norm and conditioning on $\mathcal{F}_t$, the cross term involving Δ^t vanishes by unbiasedness. Hence,

$$\mathbb{E}[\|e^{t+1}\|_{\tilde{A}^*}^2 \mid \mathcal{F}_t] = \|e^t - \eta_t \tilde{A}^* \nabla F(\tilde{\boldsymbol{\theta}}^t)\|_{\tilde{A}^*}^2 + \eta_t^2 \mathbb{E}[\|\tilde{A}^* \Delta^t\|_{\tilde{A}^*}^2 \mid \mathcal{F}_t].$$

Because $\tilde{A}^* \tilde{A}^{*\dagger} \tilde{A}^* = \tilde{A}^*$,

$$\|\tilde{A}^* \Delta^t\|_{\tilde{A}^*}^2 = (\Delta^t)^\top \tilde{A}^* \Delta^t \leq \bar{\lambda}_A \|\Delta^t\|^2.$$

Therefore,

$$\mathbb{E}[\|\tilde{A}^* \Delta^t\|_{\tilde{A}^*}^2 \mid \mathcal{F}_t] \leq \bar{\lambda}_A \nu(N_0).$$

It remains to bound the deterministic term. Since $\tilde{A}^* \nabla F(\tilde{\boldsymbol{\theta}}^*) = \mathbf{0}$,

$$\|e^t - \eta_t \tilde{A}^* \nabla F(\tilde{\boldsymbol{\theta}}^t)\|_{\tilde{A}^*}^2 = \|e^t\|_{\tilde{A}^*}^2 - 2\eta_t (e^t)^\top (\nabla F(\tilde{\boldsymbol{\theta}}^t) - \nabla F(\tilde{\boldsymbol{\theta}}^*)) + \eta_t^2 \|\tilde{A}^* (\nabla F(\tilde{\boldsymbol{\theta}}^t) - \nabla F(\tilde{\boldsymbol{\theta}}^*))\|_{\tilde{A}^*}^2.$$

By strong convexity,

$$(e^t)^\top (\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*)) \geq \mu \|e^t\|^2.$$

Since $e^t \in \text{range}(\tilde{A}^*)$,

$$\|e^t\|^2 \geq \lambda_A \|e^t\|_{\tilde{A}^{*\dagger}}^2.$$

Thus,

$$(e^t)^\top (\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*)) \geq \mu \lambda_A \|e^t\|_{\tilde{A}^{*\dagger}}^2.$$

Similarly, by smoothness,

$$\begin{aligned} \|\tilde{A}^*(\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*))\|_{\tilde{A}^{*\dagger}}^2 &= (\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*))^\top \tilde{A}^*(\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*)) \\ &\leq \bar{\lambda}_A \|\nabla F(\tilde{\theta}^t) - \nabla F(\tilde{\theta}^*)\|^2 \leq \bar{\lambda}_A L^2 \|e^t\|^2 \leq L^2 \bar{\lambda}_A^2 \|e^t\|_{\tilde{A}^{*\dagger}}^2. \end{aligned}$$

Combining these inequalities yields

$$\mathbb{E}[\|e^{t+1}\|_{\tilde{A}^{*\dagger}}^2 \mid \mathcal{F}_t] \leq \left(1 - 2\eta_t \mu \lambda_A + \eta_t^2 L^2 \bar{\lambda}_A^2\right) \|e^t\|_{\tilde{A}^{*\dagger}}^2 + \eta_t^2 \bar{\lambda}_A \nu(N_0).$$

This proves the one-step recursion.

For a constant step size, iterating the recursion gives

$$\mathbb{E}\|e^t\|_{\tilde{A}^{*\dagger}}^2 \leq \rho^t \|e^0\|_{\tilde{A}^{*\dagger}}^2 + \frac{\eta^2 \bar{\lambda}_A \nu(N_0)}{1 - \rho}.$$

Using

$$\|e^t\|^2 \leq \bar{\lambda}_A \|e^t\|_{\tilde{A}^{*\dagger}}^2, \quad \|e^0\|_{\tilde{A}^{*\dagger}}^2 \leq \lambda_A^{-1} \|e^0\|^2,$$

and

$$1 - \rho = 2\eta\mu\lambda_A - \eta^2 L^2 \bar{\lambda}_A^2,$$

we obtain

$$\mathbb{E}\|e^t\|^2 \leq \frac{\bar{\lambda}_A}{\lambda_A} \rho^t \|e^0\|^2 + \frac{\eta \bar{\lambda}_A^2 \nu(N_0)}{2\mu\lambda_A - \eta L^2 \bar{\lambda}_A^2}.$$

The diminishing-step-size result follows from the standard recursion

$$u_{t+1} \leq \left(1 - \frac{a}{t} + \frac{b}{t^2}\right) u_t + \frac{c\nu(N_0)}{t^2},$$

with $a = 2\gamma\mu\lambda_A > 1$, which implies

$$u_t = O\left(\frac{1 + \nu(N_0)}{t}\right).$$

This completes the proof.

7. Managerial Implications

The managerial implications of the proposed method are grounded in two complementary perspectives: that of the platform provider and that of participants. From the platform's perspective, adopting FOAF offers improvements in scalability and efficiency, operational resilience, adaptability, and potentially accuracy, all of which can be translated into business benefits:

1. *Improved Scalability and Efficiency:* FOAF reduces reliance on a single central server for storage, aggregation, and computation by leveraging network-based communication. This design lowers operational costs and improves scalability. Data storage is decentralized, as individual devices retain user-specific latent factors instead of burdening a central server. Communication load is reduced through peer-to-peer gradient sharing, avoiding costly bandwidth demands from massive server-participant exchanges. Computation is also distributed, lessening the server's aggregation and synchronization workload, which in turn cuts infrastructure costs and accelerates processing.

2. *Enhanced Operational Resilience:* Decentralized architectures inherently offer greater fault tolerance. For example, if a single participant (user device) goes offline, the overall system can continue to operate and learn, unlike a centralized system where server failure can halt all operations. This translates to higher availability and a more robust service.

3. *Adaptability to Dynamic Environments:* FOAF’s flexible design, particularly with schemes like real-time sharing, allows for continuous, asynchronous updates that are highly responsive to emerging user preferences and trends. This agility is crucial for e-commerce and media platforms where user tastes evolve rapidly, enabling the platform to offer more timely and relevant recommendations.

4. *On-Par (or Superior) Accuracy:* As demonstrated in the main text, FOAF achieves comparable or even superior prediction accuracy compared to centralized and federated learning baselines. This indicates that platforms can maintain, or even enhance, recommendation quality without incurring the significant overheads associated with a central server.

From the participant’s perspective, FOAF provides benefits in privacy protection and communication efficiency, which foster trust and encourage engagement:

1. *Enhanced User Privacy:* First and foremost, in a FOAF-based RS, participants can choose to keep user interaction data (e.g., movie ratings, browse history) only on the participant’s local device. Only aggregated gradient information, devoid of sensitive personal identifiers, is shared with nearby peers within a sparse network, which can be either naturally defined (e.g., by geography or business structure) or algorithmically designed. This peer-to-peer, gradient-sharing mechanism allows participants to retain full control of their data while still benefiting from collaborative learning. This fundamental privacy-by-design approach also addresses growing user concerns about data exploitation and complies with increasingly stringent privacy regulations (e.g., GDPR, CCPA), allowing platforms to build stronger trust with their user base.

2. *Reduced Communication Costs:* FOAF’s compatibility with asynchronous and stochastic updates makes it particularly well-suited for real-world applications with heterogeneous data arrival rates or limited device availability. Compared with existing decentralized learning schemes, this can also reduce communication costs for the participants, especially if local-to-local communication is optimized (e.g., through the proposed designed communication networks). This makes the system more palatable for participants with limited communication bandwidth or resource-constrained devices.

References

- [1] Muhammad Ammad-Ud-Din, Elena Ivannikova, Suleiman A Khan, Were Oyomno, Qiang Fu, Kuan Eeik Tan, and Adrian Flanagan. Federated collaborative filtering for privacy-preserving personalized recommendation system. *arXiv preprint arXiv:1901.09888*, 2019.
- [2] Chaochao Chen, Ziqi Liu, Peilin Zhao, Jun Zhou, and Xiaolong Li. Privacy preserving point-of-interest recommendation using decentralized matrix factorization. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [3] Simon Funk. Netflix update: Try this at home. <http://sifter.org/~simon/journal/20061211.html>, 2006.
- [4] István Hegedűs, Gábor Danner, and Márk Jelasity. Gossip learning as a decentralized alternative to federated learning. In *Distributed Applications and Interoperable Systems: 19th IFIP WG 6.1 International Conference, DAIS 2019, Held as Part of the 14th International Federated Conference on Distributed Computing Techniques, DisCoTec 2019, Kongens Lyngby, Denmark, June 17–21, 2019, Proceedings 19*, pages 74–90. Springer, 2019.
- [5] Yehuda Koren. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 426–434, 2008.
- [6] Langming Liu, Wanyu Wang, Xiangyu Zhao, Zijian Zhang, Chunxu Zhang, Shanru Lin, Yiqi Wang, Lixin Zou, Zitao Liu, Xuetao Wei, et al. Efficient and robust regularized federated recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 1452–1461, 2024.
- [7] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [8] Andriy Mnih and Russ R Salakhutdinov. Probabilistic matrix factorization. *Advances in neural information processing systems*, 20, 2007.
- [9] Vasileios Perifanis and Pavlos S Efraimidis. Federated neural collaborative filtering. *Knowledge-Based Systems*, 242:108441, 2022.
- [10] Chuhan Wu, Fangzhao Wu, Lingjuan Lyu, Tao Qi, Yongfeng Huang, and Xing Xie. A federated graph neural network framework for privacy-preserving personalization. *Nature Communications*, 13(1):3091, 2022.